ragent Repository Reverse-Engineering Manual

This guide explains how to use ragent's `/reverse` command to reverse-engineer the purpose, architecture, and function of public GitHub repositories. The command fetches a repository's metadata, root file tree, and README via the GitHub API, then asks the currently selected LLM model to generate a synthetic "creation prompt" — a detailed prompt that, if fed to a coding agent, would reproduce the repository from scratch.

The same functionality is available as the `ragent reverse` CLI subcommand.

> **Scope:** `/reverse` slash commands, the `--tech` and `--create` flags, GitHub API interaction, synthetic prompt generation, and spec chaining. For spec management commands, see `docs/howtos/spec.md`. For research commands, see `docs/howtos/research.md`.

---

1. Purpose and capabilities

The `/reverse` command answers a common question when encountering an unfamiliar repository: **"What does this project do, and how would I build something like it?"**

Instead of manually browsing a repo's file tree, reading the README, and guessing at the architecture, `/reverse` automates the entire process:

- **Fetches repository metadata** via the GitHub REST API — description, primary language, license, star count, topics, creation date, and homepage URL.
- **Fetches the root file tree** — the top-level directory listing of the repository, including file and directory names.
- **Fetches the README** — the full rendered README content (when available).
- **Generates a synthetic creation prompt** — passes all gathered information to the currently selected LLM model and asks it to produce a comprehensive prompt that describes how to recreate the repository from scratch, including architecture, key modules, technology choices, and implementation order.
- **Optionally constrains the technology stack** — the `--tech` flag lets you specify a target technology stack so the generated prompt is tailored to that stack rather than the repository's original languages.
- **Optionally chains into spec creation** — the `--create` flag feeds the generated prompt directly into `/spec create`, auto-generating a formal specification from the reverse-engineered prompt.

What it does

- Fetches public repository data using the GitHub API (a GitHub token is required — see Section 8).
- Derives a comprehensive creation prompt that captures the repository's architecture, module structure, key features, and technology decisions.
- Streams the generated prompt into the chat window for review.
- Can chain into the spec management system to produce a formal SPEC.md.

What it does not do

- It does not clone the repository — it uses the GitHub API only.
- It does not read source files beyond the root tree listing — it does not fetch or parse individual source files.
- It does not work with private repositories unless `GITHUB_TOKEN` is set with appropriate access.

- It does not generate code — it generates a prompt that *describes* how to build code.

---

2. Quick start

Open ragent and run:

```
/reverse thrivethrough/omitme
```

The TUI will:

1. Fetch the repository metadata, root file tree, and README from the GitHub API.
2. Pass all gathered information to the currently selected LLM model.
3. Ask the model to generate a synthetic creation prompt.
4. Display the generated prompt in the chat window.

To chain directly into spec creation:

```
/reverse thrivethrough/omitme --create omitme-clone
```

This fetches the repository, generates the creation prompt, and immediately passes it to `/spec create` to produce a formal specification under `specs/omitme-clone/`.

---

3. Command syntax

3.1 `/reverse <owner/repo>`

Fetch a public GitHub repository and generate a synthetic creation prompt.

```
/reverse <repo> [--tech <stack>] [--create <name>] [--depth <N>]
```

Arguments

Argument	Required	Description
`<repo>`	Yes	Repository identifier. Accepts `owner/repo`, `github:owner/repo`, `github:<github-url>`, `gitlab:namespace/project`, `gitlab:host/namespace/project`, HTTPS URLs (`https://github.com/owner/repo`, `https://gitlab.com/ns/proj`), and SSH URLs (`git@github.com:owner/repo.git`)

Flags

Flag	Required	Description
`--tech <stack>`	No	Constrain the generated prompt to a specific technology stack (e.g. `rust`, `python`, `typescript`). Values containing spaces must be **quoted** — `--tech "Next.js + Rails"` is captured whole; unquoted values are single tokens, so hyphenated forms like `rust-axum-sqlx` also work
`--create <name>`	No	Chain into `/spec create` using the generated prompt, creating a spec under `specs/<name>/`
`--depth <N>`	No	Directory levels to fetch from the tree (1–10, default 1). See Section 10.

3.2 Input formats

The command accepts several input formats:

```
# owner/repo shorthand
/reverse thrivethrough/omitme

# Full GitHub URL
/reverse https://github.com/thrivethrough/omitme

# With technology constraint
/reverse thrivethrough/omitme --tech rust

# With spec creation
/reverse thrivethrough/omitme --create omitme-clone

# With both flags
/reverse thrivethrough/omitme --tech rust --create omitme-rust-port
```

3.3 `/reverse help`

Show the command reference.

```
/reverse help
```

---

4. What the command gathers

When you run `/reverse <owner/repo>`, the system fetches three pieces of information from the GitHub REST API:

4.1 Repository metadata

Fetched from `GET /repos/{owner}/{repo}`:

- **Name and full name** — e.g. `thrivethrough/omitme`
- **Description** — the repository's short description
- **Primary language** — the dominant programming language
- **License** — the detected license (e.g. MIT, Apache-2.0)
- **Star count** — number of GitHub stars
- **Topics** — repository topics/tags
- **Creation date** — when the repository was created
- **Homepage URL** — the repository's homepage (if set)
- **Default branch** — e.g. `main` or `master`

4.2 Root file tree

Fetched from `GET /repos/{owner}/{repo}/contents`:

The top-level directory listing of the repository, including all files and directories at the root level. This gives the LLM a structural overview of the project layout — enough to infer the architecture, build system, and module organisation without reading individual files.

4.3 README content

Fetched from `GET /repos/{owner}/{repo}/readme`:

The full README content, rendered as markdown. The README typically contains the project overview, installation instructions, usage examples, and architecture notes — the richest single source of information about a repository's purpose and design.

---

5. The synthetic creation prompt

The core output of `/reverse` is a **synthetic creation prompt** — a natural-language description of how to recreate the repository from scratch.

The LLM receives the repository metadata, root file tree, and README, and is instructed to produce a prompt that:

1. **Summarises the project's purpose** — what the project does and who it is for.
2. **Describes the architecture** — the high-level module structure, key components, and how they interact.
3. **Identifies the technology stack** — languages, frameworks, libraries, build tools, and runtime dependencies.
4. **Outlines the implementation order** — a suggested sequence for building the project, starting with foundational modules and progressing to higher-level features.
5. **Highlights key design decisions** — notable architectural choices, patterns, and trade-offs visible from the file tree and README.

The generated prompt is designed to be fed to a coding agent (like ragent itself) to reproduce the repository's functionality from scratch.

Example generated prompt structure

```
## Project: omitme - Privacy-first error tracking
```

```
### Overview

omitme is a privacy-first error tracking service that captures application
errors without collecting personally identifiable information (PII). The
project is built in Rust and uses a client-server architecture with a
WebSocket-based real-time event stream.

### Architecture

The project consists of the following top-level modules:

- `src/server/` - Axum-based HTTP server handling event ingestion
- `src/client/` - SDK libraries for Rust, TypeScript, and Python
- `src/storage/` - SQLite-backed event store with retention policies
- `src/analytics/` - Aggregation and dashboard query engine
- `src/web/` - Static dashboard frontend (HTML/CSS/JS)

### Technology Stack

- **Language:** Rust (edition 2024)
- **Web framework:** Axum
- **Database:** SQLite via rusqlite
- **Real-time:** tokio-tungstenite (WebSockets)
- **Serialization:** serde + serde_json

### Implementation Order

1. Core data model and SQLite schema (`src/storage/`)
2. Event ingestion API endpoint (`src/server/`)
3. WebSocket event stream (`src/server/`)
4. Client SDKs (`src/client/`)
5. Analytics and aggregation queries (`src/analytics/`)
6. Dashboard frontend (`src/web/`)

### Key Design Decisions

- PII stripping happens client-side before transmission
- Events are stored with a configurable retention period
- The dashboard is a static site with no server-side rendering
```

---

6. The `--tech` flag

By default, the generated prompt reflects the repository's original technology stack. The `--tech <stack>` flag constrains the prompt to a specific target stack, making it useful for planning a port or rewrite. Multi-word stacks are supported when quoted: single and double quotes group the value into one token and the quote characters are stripped.

How it works

When `--tech` is supplied, the LLM is instructed to:

- Describe how to recreate the repository's *functionality* using the specified technology stack.
- Map the original architecture to equivalent components in the target stack.
- Note where the target stack differs from the original and what adjustments are needed.

Examples

Port a Python project to Rust:

```
/reverse example/flask-api --tech rust
```

Port a Node.js project to Python:

```
/reverse example/express-server --tech python
```

Port a Go project to TypeScript:

```
/reverse example/go-microservice --tech typescript
```

Specify a more detailed stack:

```
/reverse example/legacy-java-app --tech "Rust with axum and SQLx"
```

---

7. The `--create` flag

The `--create <name>` flag chains the reverse-engineering output into `/spec create`, auto-generating a formal specification from the reverse-engineered prompt.

How it works

1. `/reverse` fetches the repository and generates the synthetic creation prompt.
2. The generated prompt is passed as the feature description to `/spec create <name>`.
3. The spec system creates `specs/<name>/SPEC.md` (EARS requirements), `specs/<name>/PLAN.md` (implementation plan), and `specs/<name>/TESTPLAN.md` (manual test plan).

Examples

Reverse-engineer a repo and create a spec:

```
/reverse thrivethrough/omitme --create omitme-clone
```

Reverse-engineer with a tech constraint and create a spec:

```
/reverse thrivethrough/omitme --tech rust --create omitme-rust-port
```

Reverse-engineer a large project into a spec:

```
/reverse tokio-rs/tokio --create tokio-study
```

What you get

After `--create` finishes, the spec directory contains:

```
specs/omitme-clone/
   SPEC.md          # EARS requirements derived from the creation prompt
   PLAN.md          # Implementation plan with task table
   TESTPLAN.md      # Manual test plan with test cases
```

You can then use the standard `/spec` commands to validate, plan, and implement the spec (see `docs/howtos/spec.md` for details).

---

8. API interaction

GitHub authentication

A GitHub token is a **prerequisite**: `/reverse` refuses to run without one and reports "No GitHub token configured. Run `/github login` to authenticate, then re-run `/reverse`." The token is resolved via `/github login` (OAuth device flow, stored in `~/.ragent/github_token`) or the `GITHUB_TOKEN` environment variable. Unauthenticated GitHub API access allows only 60 requests per hour per IP; an authenticated token raises this to 5,000 requests per hour.

```
export GITHUB_TOKEN="ghp_your_token_here"
```

The token does not need any special scopes for reading public repositories. A fine-grained token with read access to public repositories is sufficient.

GitLab authentication

GitLab API calls require a personal access token. The token is resolved in priority order:

1. `GITLAB_TOKEN` environment variable
2. `ragent.json` configuration
3. Encrypted credential database (configured via `/gitlab setup`)

For self-hosted GitLab instances, the host is resolved in priority order: explicit host in the repo identifier (`gitlab:host/namespace/project`), then the `GITLAB_URL` environment variable, then `https://gitlab.com`.

```
export GITLAB_TOKEN="glpat-your_token_here"
export GITLAB_URL="https://gitlab.example.com"
```

Rate limiting

If the API rate limit is exceeded, the command will report an error. Wait for the rate limit window to reset (typically 1 hour) or set `GITHUB_TOKEN` for higher limits.

When using `--depth` for recursive tree fetch, each subdirectory level adds additional API calls (one per subdirectory). Use a token for deeper trees to avoid hitting the rate limit.

Error handling

Error	Cause	What to do
`repository not found`	Invalid owner/repo or private repo	Check the spelling; set `GITHUB_TOKEN` for private repos
`rate limit exceeded`	Too many API requests	Wait for reset or set `GITHUB_TOKEN`
`README not found`	Repository has no README	The command continues with metadata and tree only
`network error`	Connection issue	Check network connectivity and retry
`gitlab token missing`	No GitLab token configured	Set `GITLAB_TOKEN` env var or run `/gitlab setup`
`invalid depth`	`--depth` value out of range	Use a value between 1 and 10

> **Silent degradation on GitHub fetch failures:** metadata/tree fetch failures on the GitHub path are swallowed (`unwrap_or_default()`), so an API error produces an empty context fed to the LLM rather than an error message — the `repository not found` / `rate limit exceeded` / `network error` messages only surface on the **GitLab** path. If the generated prompt looks thin, re-run after checking your token and rate limit.

---

9. CLI equivalents

The TUI `/reverse` command is the primary entry point; the equivalent `ragent reverse` CLI subcommand is not currently registered in the clap CLI (`run`, `serve`, `session`, `memory`, `auth`, `models`, `config`, `research` are the available subcommands). The examples below describe the intended CLI surface and will become accurate when the subcommand ships:

```
# Basic reverse-engineering
ragent reverse thrivethrough/omitme

# With technology constraint
ragent reverse thrivethrough/omitme --tech rust

# With spec creation
ragent reverse thrivethrough/omitme --create omitme-clone

# Full GitHub URL
ragent reverse https://github.com/thrivethrough/omitme

# Both flags
ragent reverse thrivethrough/omitme --tech rust --create omitme-rust-port
```

The CLI command prints the generated prompt to stdout, making it easy to pipe into other tools or save to a file:

```
# Save the generated prompt to a file
ragent reverse thrivethrough/omitme > omitme-prompt.md

# Pipe through jq for structured output
ragent reverse thrivethrough/omitme --tech rust 2>&1 | tee omitme-rust.md
```

---

10. The `--depth` flag

By default, `/reverse` fetches only the root-level file tree (depth 1). The `--depth <N>` flag controls how many levels of subdirectories are expanded.

Depth	What you get
1 (default)	Root-level files and directories only
2	Root + one level of subdirectories
3	Root + two levels of subdirectories
N (max 10)	Root + N-1 levels of subdirectories

Deeper trees give the LLM more context about the project's module structure, which improves the quality of the generated creation prompt — especially for large repositories with deep nesting. However, each additional level requires more API calls (one per subdirectory), so use a token for higher rate limits when fetching deep trees.

Examples

```
# Default depth (root only)
/reverse thrivethrough/omitme

# Two levels deep
/reverse thrivethrough/omitme --depth 2

# Maximum depth for a large project
/reverse torvalds/linux --depth 5 --tech rust
```

---

11. End-to-end examples

Example 1: Understand an unfamiliar project

You find an interesting repository and want to understand what it does:

```
/reverse thrivethrough/omitme
```

The generated prompt gives you a comprehensive overview of the project's purpose, architecture, and technology stack — without needing to browse the code yourself.

Example 2: Plan a port to a different language

You want to port a Python project to Rust:

```
/reverse example/flask-api --tech rust
```

The generated prompt describes how to recreate the Flask API's functionality using Rust, mapping each component to its Rust equivalent (e.g. Flask → Axum, SQLAlchemy → SQLx, Celery → Tokio tasks).

Example 3: Create a spec from a reference implementation

You want to create a formal spec based on an existing open-source project:

```
/reverse tokio-rs/tokio --create tokio-study
```

This generates a spec under `specs/tokio-study/` with EARS requirements, an implementation plan, and a test plan — all derived from the reverse-engineered creation prompt.

Example 4: Study a well-architected project

You want to study how a well-known project is structured:

```
/reverse burntsushi/regex
```

The generated prompt breaks down the regex crate's architecture, module structure, and key design decisions, giving you a learning roadmap.

Example 5: Plan a rewrite with a specific stack

You want to rewrite a legacy application using a modern stack:

```
/reverse example/legacy-monolith --tech "Rust with axum, SQLx, and Redis"
```

The quotes are required here: an unquoted value would stop at the first space. The generated prompt maps the legacy monolith's functionality to a modern Rust microservice architecture with specific crate recommendations.

Example 6: Reverse-engineer and immediately implement

Combine `/reverse` with `/spec impl` for a full pipeline:

```
# Step 1: Reverse-engineer and create a spec
/reverse thrivethrough/omitme --tech rust --create omitme-rust

# Step 2: Validate the generated spec
/spec validate omitme-rust

# Step 3: Preview the implementation plan
/spec impl omitme-rust --dry-run

# Step 4: Implement
/spec impl omitme-rust
```

Example 7: Compare two implementations

Reverse-engineer two competing projects to compare their approaches:

```
/reverse project-a/repo --create study-a
/reverse project-b/repo --create study-b
```

Then compare the generated specs to understand the architectural differences between the two projects.

Example 8: Generate a prompt for a specific use case

You want a creation prompt focused on the testing infrastructure of a project:

```
/reverse example/well-tested-app --tech rust-proptest
```

The generated prompt emphasises how to recreate the project's testing approach using the specified testing tools.

Example 9: Reverse-engineer a GitLab repository

```
# Self-hosted GitLab with nested namespace
/reverse gitlab:gitlab.example.com/group/subgroup/project --depth 3

# GitLab.com with spec creation
/reverse gitlab:my-namespace/my-project --create my-project-clone

# Full GitLab URL
/reverse https://gitlab.com/my-namespace/my-project --tech python --create my-py-port
```

---

12. Tips for good results

- **Use specific tech stacks.** `--tech rust` is good, but unquoted values end at the first space — quote multi-word stacks (`--tech "Rust with axum and SQLx"`) or use hyphenated descriptors like `--tech rust-axum-sqlx` for more targeted prompts.
- **Review the generated prompt before acting.** The synthetic prompt is a starting point — review it for accuracy and adjust before feeding it to a coding agent.
- **Use `--create` for structured workflows.** Chaining into `/spec create` gives you a formal spec, plan, and test plan — much more actionable than a raw prompt.
- **Set `GITHUB_TOKEN` for frequent use.** A token is required to run the command at all; the unauthenticated API allows only 60 requests/hour, authenticated is 5,000/hour.
- **Use full URLs for clarity.** `/reverse https://github.com/owner/repo` is unambiguous; `/reverse owner/repo` is faster to type.
- **Combine with `/spec` commands.** After `--create`, use `/spec validate`, `/spec plan`, `/spec tasks`, and `/spec impl` for a complete spec-driven workflow.
- **Use `--tech` for port planning.** When planning a port, always specify the target stack so the prompt maps to the right technologies.
- **Try different models.** The generated prompt quality depends on the selected LLM model. Try different models (e.g. Claude, GPT-4) to compare output quality.
- **Study well-architected projects.** Use `/reverse` on projects you admire to learn about their architecture and design patterns.

---

13. Troubleshooting

Symptom	Likely cause	What to do
`repository not found`	Invalid owner/repo or private repo	Check spelling; set `GITHUB_TOKEN` for private repos
`rate limit exceeded`	Too many unauthenticated API calls	Set `GITHUB_TOKEN` environment variable
`README not found`	Repository has no README	The command continues with metadata and tree only
Empty or short prompt	LLM model not configured or down	Check provider setup with `/models`
`network error`	Connection issue	Check network connectivity and retry
`invalid repository format`	Malformed input	Use `owner/repo` or a full GitHub URL
`spec already exists`	`--create` target name already used	Choose a different name or delete the existing spec
Prompt mentions wrong technologies	`--tech` not specified or too vague	Use a more specific `--tech` value

---

14. Workflow integration

The `/reverse` command integrates with ragent's spec management system to provide a complete reverse-engineering-to-implementation pipeline:

```
/reverse <owner/repo>


  GitHub API fetch
  (metadata + tree
   + README)



  LLM generates
  creation prompt


               --create <name>
  Display prompt                      /spec create
  in chat window                                generates SPEC.md,
                                    PLAN.md, TESTPLAN.md
```

```
/spec validate
/spec plan
/spec impl
```

Full pipeline example

```
# 1. Reverse-engineer a project
/reverse thrivethrough/omitme --tech rust --create omitme-rust

# 2. Validate the generated spec
/spec validate omitme-rust

# 3. Generate task list
/spec tasks omitme-rust

# 4. Preview implementation
/spec impl omitme-rust --dry-run

# 5. Implement
/spec impl omitme-rust
```

This pipeline takes you from an unfamiliar GitHub repository to a running implementation in five commands.

End of manual.